Design and develop a distributed Hotel booking application using Java RMI. A distributed hotel booking system consists of the hotel server and the client machines. The server manages hotel rooms booking information. A customer can invoke the following operations at his machine i) Book the room for the specific guest ii) Cancel the booking of a guest.

Code :

```
import java.rmi.*;
import java.rmi.registry.*;
import java.rmi.server.*;
import java.util.*;

// ================================
// MAIN CLASS (ALL IN ONE FILE)
// ================================
public class HotelRMI {

    // ================================
    // 1. REMOTE INTERFACE
    // ================================
    public interface HotelService extends Remote {
        String bookRoom(String guestName) throws RemoteException;
        String cancelBooking(String guestName) throws RemoteException;
    }

    // ================================
    // 2. SERVER IMPLEMENTATION
    // ================================
    public static class HotelServiceImpl extends UnicastRemoteObject implements HotelService {
        private static final long serialVersionUID = 1L;

        HashMap<String, Integer> bookings = new HashMap<>();
        int roomNo = 1;

        protected HotelServiceImpl() throws RemoteException {
            super();
        }

        public String bookRoom(String guestName) throws RemoteException {
            if (bookings.containsKey(guestName)) {
                return "Already booked for " + guestName;
            }
            bookings.put(guestName, roomNo++);
            return "Room booked for " + guestName + ", Room No: " + bookings.get(guestName);
        }
```

```
    public String cancelBooking(String guestName) throws RemoteException {
    if (!bookings.containsKey(guestName)) {
      return "No booking found for " + guestName;
    }
    bookings.remove(guestName);
    return "Booking cancelled for " + guestName;
  }
}
// ================================
  // 3. SERVER START
  // ================================
  public static class Server {
      public static void start() {
          try {
              HotelService service = new HotelServiceImpl();

              Registry registry = LocateRegistry.createRegistry(1099);
              registry.rebind("HotelService", service);

              System.out.println("     Server Started...");
              System.out.println("Server is running. Press Ctrl+C to stop.");

              // Keep the server running
              synchronized (Server.class) {
                  Server.class.wait();
              }
          } catch (Exception e) {
              e.printStackTrace();
          }
      }
  }

  // ================================
  // 4. CLIENT
  // ================================
  public static class Client {
      public static void start() {
          Scanner sc = new Scanner(System.in);
          try {
              Registry registry = LocateRegistry.getRegistry("localhost", 1099);
              HotelService service = (HotelService) registry.lookup("HotelService");

              while (true) {
                  System.out.println("\n1. Book Room");
                  System.out.println("2. Cancel Booking");
                  System.out.println("3. Exit");
                  System.out.print("Enter choice: ");

                  int choice = sc.nextInt();
                  sc.nextLine();
```

```
            if (choice == 1) {
            System.out.print("Enter name: ");
            String name = sc.nextLine();
            System.out.println(service.bookRoom(name));

        }   else if (choice == 2) {
                    System.out.print("Enter name: ");
                    String name = sc.nextLine();
                    System.out.println(service.cancelBooking(name));

                } else {
                    break;
                }
            }

            } catch (Exception e) {
                e.printStackTrace();
            } finally {
                sc.close();
            }
        }
    }

    // ================================
    // 5. MAIN METHOD
    // ================================
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        try {
            System.out.println("1. Start Server");
            System.out.println("2. Start Client");
            System.out.print("Choose: ");

            int choice = sc.nextInt();

            if (choice == 1) {
                Server.start();
            } else {
                Client.start();
            }
        } finally {
            sc.close();
        }
    }
}
```

HotelRMI.java 9, U java

```
PS C:\Users\Rohan\Desktop\BE Practicals\DC>  java HotelRMI
1. Start Server
2. Start Client
Choose: 1
? Server Started...
Server is running. Press Ctrl+C to stop.
```

PROBLEMS 9 OUTPUT TERMINAL PORTS GITLENS

```
PS C:\Users\Rohan\Desktop\BE Practicals\DC>  java HotelRMI
Choose: 2

1. Book Room
2. Cancel Booking
3. Exit
Enter choice: 1
Enter name: Rohan Thite
Room booked for Rohan Thite, Room No: 1

1. Book Room
2. Cancel Booking
3. Exit
Enter choice: 2
Enter name: Rohan
No booking found for Rohan

1. Book Room
2. Cancel Booking
3. Exit
Enter choice: 2
Enter name: Rohan Thite
Booking cancelled for Rohan Thite

1. Book Room
2. Cancel Booking
3. Exit
Enter choice: 3
PS C:\Users\Rohan\Desktop\BE Practicals\DC>
```